图像绘制

`drawImage(image, x, y)`

在画布中绘制图像。（API具体使用上可查阅使用图像 - Web API 接口参考 | MDN）

使用示例

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

// 加载图像
const image = new Image();
image.crossOrigin = 'anonymous';
image.onload = () => {
  console.log('image loaded! ')

  canvas.width = image.naturalWidth;
  canvas.height = image.naturalHeight;

  ctx.drawImage(image, 0, 0, canvas.width, canvas.height);
}
// 图像链接
image.src = 'https://i.postimg.cc/Xvw8CFhS/fat-cat-small.png';
```

进阶使用

① 不仅仅是图像链接，还可以用另一个 `canvas` 作为 `drawImage` 的图像源。

```
ctx.drawImage(otherCanvas, 0, 0);
```

② 请说说下面的写法会造成什么问题：

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

const image = new Image();
image.crossOrigin = 'anonymous';
image.src = 'https://i.postimg.cc/Xvw8CFhS/fat-cat-small.png';
```

```
// ...通过setTimeout模拟执行了额外异步操作
setTimeout(() => {
  image.onload = () => {
    console.log('image loaded! ')
    canvas.width = image.naturalWidth;
    canvas.height = image.naturalHeight;

    ctx.drawImage(image, 0, 0, canvas.width, canvas.height);
  }
}, 0);
```

解答：

`onload` 方法将不会调用，因为 `onload` 方法调用的时机是在图像加载完成的那一刻调用，这里一段时间过后，图像可能过了加载完成的时机，所以不会调用。

这种问题我之前在热更新的项目本地开发中遇到，因为图像已经被加载和缓存，实际上他已经过了加载完成时机，就不会再执行 `onload` 回调了。

所以最佳实践是习惯性将 `onload` 调用写在 `src` 赋值之前。

③ 阻止图像因"平滑服务"而变得模糊

图像加入画布是会自动使用平滑算法，如下：

如果要阻止使用平滑算法，需要设置 `ctx.imageSmoothingEnabled = false` ，如下：

这里需要注意的是： `imageSmoothingEnabled` 仅在使用 `drawImage` 绘制图像的场景下有效。

如果你想阻止 `canvas` 直接绘制（比如路径描边等）时的平滑服务，那是做不到的。

浏览器页面显示画布本身也会添加一层平滑，如果我们给 `canvas` 节点添加 `imageRendering: pixelated;` 样式，我们可以看到画布本身的像素效果，也就是下载图像后的效果。